

2026-05-25 16-01-29

Hey, good afternoon guys. Hi Madhukar. Hi Madhukar.

Good afternoon sir. Good afternoon sir. Hello everyone.

Hi Rajesh. Hello Suman. Hello, good afternoon.

Hi everyone. Good afternoon sir. Hello.

Can we start? Yeah. I think we can get started. Yes.

Is my screen visible? Yes. So, in the previous meeting, we discussed the high level design of our system along with the UI mockup to understand how the overall experience will look from the end user's perspective. The agenda for today's meeting is to go deeper into the internal working of our work trial suite.

So, we will be explaining each major component of the system using detailed sequence diagrams and we will be discussing how we address the challenges mentioned in the problem statement after incorporating all your suggestions and feedbacks during the previous meetings. So, let us first briefly look at everything we will be covering today. First, we will begin with the live bug list view and briefly how our system fetches from multiple connected external systems in parallel, how we optimize performance using caching and how filtering and search are handled efficiently across different platforms.

After that, we will move to the complete triage pipeline and understand what happens when an engineer clicks the triage button on one particular bug. Here we will discuss how our four agents collaborate with each other throughout the workflow. This includes the context fetch agent which gathers the complete bug context, the cross system fetch agent which identifies similar bugs across other systems, the enrichment agent which fetches relevant knowledge-based articles and the AI synthesis agent which combines all the collected information and generates an AI-based summary along with a confidence score.

Next, we will discuss the connector configuration architecture and how we will be using the adapter pattern to integrate with multiple external systems while keeping the system extensible. Lastly, we will cover how we handle incremental updates using audit logs and change log APIs from external systems. So, throughout the presentation we will go through detailed sequence diagram for each component and understand the architectural decisions.

Now, we will start with the first section which is the live bug list view. So, looking at how we are handling the data. Hello, would you mind recording this session so that we can refer it later as well? Yes, we are recording.

Okay, thank you. Please go ahead. So, looking at how we are handling the data aggregation and the caching layer for the bug dashboard.

The big bottleneck here is that pulling live tickets across multiple external trackers like Zira, Bugzilla and GitHub that is basically incredibly slow. So, to solve this problem we went with an async parallel fetch pattern. So, we are using FastAPI to run concurrent workers.

Redis is a quick cache store and Postgres for persistent audit logging. So, the scenario 1 starting with the first sequence diagram. This represents a complete cold cache.

Basically, what happens when an engineer loads up the dashboard for the very first time. So, basically what happens is that FastAPI checks Redis for the data right away because the cache is empty. It comes back completely blank.

Now, instead of slowing things down by making those external API calls one after the other, FastAPI fires off the request to multiple external systems connected at the same time and as soon as those raw payloads come back our FastAPI backend normalizes them into a single unified ticket data schema and the combined list which is all the total bugs is saved straight back to Redis with a short 2-minute TTL. The second sequence diagram shows the exact same request but when the cache is already warmed up. Now, if someone opens the dashboard or hits refresh within that same 2-minute window.

So, what exactly happens is FastAPI checks Redis and instantly gets the pre-saved ticket list back. So, because we skipped the external networks and the third-party APIs completely, this time the response time drops from seconds down to about milliseconds and FastAPI does a quick sorting and pagination in memory and it just sends the entire data right to the frontend. Now, in the third diagram which covers what happens the moment an engineer filters the UI by typing a term, let's say storage controller, basically into the search bar.

Now, instead of sending a heavy search query down to each individual tracking system, we are using the same warm cache sitting inside Redis. FastAPI takes all the open bugs from Redis instantly and then filters them directly by looping through them and matching the titles or IDs. And the final scenario which is in the sequence diagram 4 that demonstrates the partial cache mishandling.

Now, let's say there is a scenario that engineer filters the bug by some severity, let's say P1. Now, FastAPI first takes Redis for a specialized filtered cache, specifically for P1 bugs. Now, even though Redis already contains the general cache for all open bugs, but it is a possibility that it may not yet contain the exact filtered cache for P1 only.

So, it returns a miss for that specific key. Now, because of this partial miss, what really happens is FastAPI performs a targeted API call to the system requesting only for that P1 priority bugs and then the system returns the required bugs which are again normalized, stored back into Redis as a new filtered cache and they are finally returned to the frontend. So, over to Anuj for the next part.

Now, moving to our overall architecture diagram, we have taken down to 6 parts. So, this is diagram 1 where an engineer interacts with our FastAPI which receives the trials request. The trials request is then pushed into the Kafka which acts as our event bus and then Kafka feeds into the orchestrator which is the brain of our system.

It manages the workflow and coordinates 4 agents, that is context-switch, raw system enrichment and AI synthesis agents. These agents interact with 2 external layers, that is firstly LLM and another one is STERA, DATA, VOXELA, CONTROL and CUSTOM PORTAL. For

reliability, we maintain temporary workflow check in pipeline context store observations audit log and updates back to the engineer using the WebSocket.

Now, coming to the actual workflow, when the engineer clicks trials it validates through the JWT token, generates a unique case ID and publishes a trials request event into the Kafka. At the same time, the browser opens a WebSocket connection and then publishes into the complete WebSocket. After this, the orchestrator consumes the Kafka event and creates a pipeline context store which acts as crash-safe working memory for the workflow.

Next on the context-switch agent, this phase runs first because every later stage output depends on this. The context-switch agent retrieves complete bug data from JIRA, including logs, commands, metadata, timestamps. It also fetches linked customer cases from the customer portal.

The data is then normalized into the standard Qt format. Then the orchestrator stores this information into the pipeline context and immediately streams panel 1 bug data back to the engineer through WebSocket. Next comes our next phase, the parallel intelligence layer in which there are two agents running simultaneously, the cross-system agent and enrichment agent.

The cross-system agent first asks LLM to generate an optimized semantic search query for the bug. That query is then fired through the Bugzilla, GitHub, and JIRA network. The return candidate issues are again solved using LLM and low confidence methods before pushing out to the panel 2 related issues.

At the same time, the enrichment agent is also working so the LLM decides what to search. We execute the search, return the results back to the model, and refine the query again until enough the context is valid. The enrichment knowledge is streamed as panel 3 link context.

All of this context is continuously stored in the pipeline context which can be used at the next phase. Once both parallel agents start complete, then the AI agent starts. It assembles the complete context, the original ticket ID, related issues, A-B articles, and customer cases.

This full package is sent to the LLM, then with temperature 0 to keep the output deterministic. The model records structured outputs such as severity, confidence, truth causes, and recommended actions. We validate this using a strict identic schema before streaming the final AI summary panel back to the engineer.

Finally, the last step of the cleanup phase, we insert a lightweight audit penalty containing only system observations such as severity, system observations, and execution duration. Then the temporary pipeline context row is deleted, so no external bug data is permanently stored in our system. Finally, we commit task offset marking.

The workflow is successfully completed. This is our overall architecture which updates our complete pipeline. This is a diagram of combining all the fixed items initializing with phase 0, then phase 1 context agent, then our two parallel agents, way synthesis, and last are cleanup processes.

So, moving on to the home. Yeah, so Yeah, so now we see how we can add a connection and disable the connection starting with add connection. Am I audible now? Yes.

So, starting with add connection, the admin fill a form with system type, base URL, token, project key, ticket prefix, and send it to FastAPI. FastAPI receive this and first map the system type to a connector class. So, in this, like 0 become 0 connector.

This is our adapter pattern. Every source system has its own connector class, but they all follow the same standard interface. Now, before saving anything, FastAPI creates a temporary connector interface instance and fires a live test call to actual external system testing if the URL and token are valid.

Two outcomes here, if the test fails, we immediately return bad request to the admin. Nothing is saved anywhere. If the test pass, we write two things separately.

First, we insert the connection config into the source registry. Second, the token goes into the encrypted token vault. These two things live in completely different systems for security reasons.

The admin get a success response and the connection is immediately live. No restart is needed. Now, we see disabled connection.

The admin sends a delete request, but we do not actually delete the row. We only mark it as disabled. The row stays permanently in the source registry.

The next get all enabled call simply skip rows where enabled is false. The source disappears from the active system instantly with no restart. Shivansh? Quick question.

Any reason why you decided not to delete? Just to understand. So, we do not delete because for hard delete, delete.tryHST so for... Yeah, it's just to ensure that when we call the get all enabled connectors. So, in future, only the connectors that have been set to true gets called.

The permanent delete is not generally preferred in such systems because if we need to enable it again in the future, we could just toggle it to true and without any system downtime, we could initiate that connection also. Yes, it also breaks the audit trail as well like existing audit log entries which will be referencing that particular source would then also be later used in future. That's the point.

Okay. So, this connector registry and source registry, those are the components that is developed as part of this here? Yes, we will be writing these classes, Python classes where we will be telling how we will be talking to external systems. So, every source registry row represents what connector is there in what connector class we are using and how does it maps to the external system.

So, how the connector registry and source registry are different? Connector registry is the concept. Source registry is the table that we are storing in the Postgres database. The name of the table is source registry.

Table. Okay. It's difficult to understand between the connector registry and source registry.

Table itself will not become an entity here. Okay. So, you are basically designing a registry and you are interacting with that registry and that registry has some persistence.

Yes. The persistence is the connector classes like those API tokens. Where are the Python classes which maps to that particular connector like we have Zeroconnector.

So, Zeroconnector class will map in this particular connector registry. So, those mappings are done in these registries. Okay.

What about token vault that you are showing? What is that system would look like? It's for security storing those tokens and secrets like the API tokens that could be violated and for security reasons, we are storing them in a token vault. That could be anything like AWS Secret Manager or HashiCorp vault. That's a concept that we are using here.

Okay. What are you planning right now for your test system that you are going to develop? What are you using for that vault? We are planning to use HashiCorp vault for the test system. Initially, that's what we are planning to use.

Do you have access to HashiCorp vault? We will first start with storing the credentials in the environment variables for the test purposes. Once we have tested them on our system, we will be further looking into how we could make it more secure and store it in token vault. We will be primarily starting with saving it in environment variables.

Okay. So, if you want to use a HashiCorp vault, it comes with a license. For that one, there is an open-source alternative called OpenBAO.

You should use that. Oh, okay. Can you tell the name again? OpenBAO.

B-A-O. Open... B-A-O. Let me just put it in the chat.

Oh, it looks like turned off. Chat is not there in this. Okay.

So, search for OpenBAO. You should get it. Yes, sir.

And also, when you say add connection and disable connection, is there any way of glossary that you are maintaining? What do you mean by connection? There is a base connector class which consists of four. I will tell you how we are connecting to external systems using that after. No, my question is not that.

My question is do you maintain a glossary of all the terms, whatever that you are using? Because here, you are mentioning the use case like add connection and disable connection. So, how does one understand what do you mean by connection? Currently, no. The exact glossary, we are not mapping, but we will document this.

One needs to understand what this entity is. As part of the whole system, what does connection mean? What does connector registry mean? Then, I will be able to connect between the different subsystems which are available as part of this whole system. So, we will document those glossary terms, all that we discussed today and we will share.

Generally, you should define a system in general and then what are the subsystems within it and then you will deep dive into the individual sequence diagrams there. There, you will start bringing in different entities and then how the interaction that's going to happen between these entities. Without that, it's very difficult to connect and say okay, all of a sudden if I say add connection, so what am I talking about? Okay? Okay.

Okay, go ahead. So, the next thing we wanted to talk about is how we are handling incremental changes. In our previous approach, we were doing hard API calls every time we wanted to see if there is a change or not.

So, it was really taxing on our system. So, what we are proposing is a two-tier hybrid approach. Firstly, a lightweight API call to see if there is something changed by cross-referencing an audit log that we are maintaining and then firing a second API call if there is a change in the system.

So, there can be three scenarios that can be considered. The first one, if there is a change detected. So, what we can do is let's say there is a bug called Storage 441.

The FastAPI service pulls the audit log and checks what's the last updated status of this bug. Then we can make an external lightweight call to our Jira system asking what is the current updated status of the bug. If it says that the updated system is changed like before it was April 14th, now it's April 15th.

So, we can authorize our second hard API call to fetch the full change log and get the relevant changes that are happening. The second scenario can be that there is no change in the bug. So, what we'll do is we'll cross-reference the audit log and the FastAPI and the API calling will check against it.

If there is no change detected in the updated column, we'll return that no significant triage is needed. The third scenario can be that we don't have the current bug in our audit log. That means it hasn't been triaged.

So, we entirely skip our API calls and return a flag that this bug needs to be triaged in our system. So, using this approach we are decoupling the UI responsiveness and hard API calling saving compute and network resources. Given that store 440 or whatever that example that you used, how do you know it is pertaining to the Jira system? Like when we are fetching it firstly from the Jira, we are assigning an internal bug ID to it from our system.

So, when we are cross-referencing it from our audit log, we can know that this current bug, like let's say the internal ID is bug 01. So, when we are fetching this bug again from the Jira system, when there is a change, we can cross-reference it that the timestamp has been updated, the severity has changed. First thing is let's say now you zeroed on STOR 4401 or whatever bug ID, right? So, from that one, how did you arrive at this particular bug is being tracked in Jira and not in any other system? Like when we are adding connections like GitHub repositories and Jira projects, we are specifying that this project is being tracked in Jira and this project is tracked in GitHub.

And the prefix that this uses, like Jira uses a specific set of prefixes, we are assigning GitHub issues separate prefixes. So, from that we are tracking that this bug arrived from Jira or GitHub or Bugzilla. Okay.

I don't think that is the way to track it. Okay. Also, adding to Shivan's point, I would like to say that our system takes the source registry table also in Postgres.

So, whenever we set up any connection via the token settings, we define that project prefix and through that project prefix, it can be mapped which system that particular bug belongs to. So, this is what we are exactly using here in our system to map what a particular bug belongs to, which system. Okay.

Go ahead. We will discuss further. Okay.

That's all we have for today's presentation. Okay. So, Madhuban and Rajesh, do you have anything? For me, considering that we will have multiple bugs which are tracked through multiple bug systems.

So, are we really asking them to go by the ID and the pattern based on which you will map it to the respective bug's repository? It should not be specific to the ID, right? So, it could be like, there could be duplicate, like the same ID could be there in either of the bug tracking system, right? Right. So, it should not ideally be based on just the ID. It could be that like we have information in terms of what are the bug databases that we can access and then a particular ID should be specific to that particular bug tracking system.

So, that should be known beforehand, isn't it? Because it's not a guessing game, right? We don't have to guess. It's already known that, okay, this particular ID belongs to which tracking system. So, we don't have to have, we shouldn't be needing to dynamically determine that.

So, if there is a report, like let's say there is a customer issue reported, so then the customer facing portal that will have a certain ID that there will be engineering facing portal which have a corresponding ID and so on. So, it's a known information. So, you just have to figure out that from the input itself and then just act upon it.

And the URLs to access those bug tracking systems should be made configurable in your portal and you should be able to just directly access them based on this prior information which you should assume that it's a known information. Yes, that's what we are doing. When we are fetching the information from one particular external system, when we are entering that but we are also taking that source systems into our table like this particular JIRA ID this particular bug ID is associated with this particular bug system.

So, if in the UI mockup, Anish could you please go on that site where we are configuring connections. At the configuration yes. In the UI mockup.

Okay. Yes. So, here we are managing all our connections.

We could dynamically add these base URLs when we are to add any connections and whenever we are fetching a bug, we are also entering that. We are also keeping the record of that source. Okay.

So, this is one use case where we know that there is already like a bug ID that has been mentioned let's say in one of the portal and it's references another portal's bug ID. But there can be cases where we will have to figure out dynamically right where we want to find similar bug matches or we want to find out the corresponding bug IDs from various other portals like from CFI portal we get the let's say a log information or logs that were generated around the time that an issue occurred and then the corresponding backtraces if it is a crash issue and use that as metadata information to be able to do like use a LLM based search and to be able to figure out whether there are similar matches and the correlated issues we will be able to find. Was that part of the scope for this particular? Could we map to the previous slide Anuj? The four panels. Has that been accommodated in the design and? Yes, that's what we discussed in one of the sequence diagrams like these related issues or the second one where we are fetching similar bugs along with their source systems their bug IDs and we are assigning a similarity score.

The way we are searching those external systems are through via LLM like the context we got from one particular bug of which we want to have similar bugs using that context we generate a search query and search all the other bug systems in parallel and fetches them along with them and assign a similarity score based on how similar they are. Okay, so that has been already taken care and it's taken into account for it. Yes, the case notes also are there in the bug context.

The link context the KB articles we will be fetching from confluence is in third panel and all this information will be correlated in the AI summary panel which will show the root causes the recommended actions based on whatever context it has gathered from the first bug context the related issues and the confluence KB articles that we could access. Okay, for the demonstration purpose what would be your inputs like from where are you where will you be getting the inputs have you planned have you done any planning around that. There might be two cases either we will be seeding the external systems with mock data or we were going to ask you if we could have a data from your site which we could see in those systems.

Rajesh or Divakar any thoughts on that I don't think we can share any authentication based URLs or any of the bug tracker systems within HPE for sure. So we'll have to figure out an alternative where they may have to synthesize or use a public facing portal bug tracking system in GitHub or something like that right. Yeah yeah so as long as you are able to just show the multiple endpoints where you can register these bugs into right so that should be a more generic design not necessarily that okay you you are going to go with a very specific JIRA system or whatever right.

So as long as you are able to design your system in such a way that I can connect with JIRA or GitHub or any other it could be any other backend. Okay so we'll demonstrate such connections and we'll try to seed some AI generated data to those systems. Or you could use a public facing open source project and see where the bugs are tracked so those would be available for you to you know utilize that would be the case instead of doing a AI generated thing use a public facing project some maybe a AI based project or whatever

open source projects that are available you pick up one of them and then look at the where the bugs are getting tracked and then try and reuse those.

But that might be associated with one bug tracker. Yeah you need to figure out like if there are such open source projects where there are more than one bug tracker system and things like that you might need to cross check that. Yeah there are plenty of open source projects which are out there in the Linux OS itself there are so many.

Okay so we'll check that out. Yeah. So any other questions? So we are is there any code that has been generated or I mean have you written up any code or we are still at the design review phase or what stage are we at right now? Finalizing the design stage like in the previous meeting we asked to validate the design along the way of creating this design have you validated your design whether it is feasible and whether it works and factors to consider pros and cons and all of that you have that is we have seen we have factored the design decisions but we will we are still trying to implement all of these like we are utilizing rock for this we are seeding mock data and we are trying to see they could fetch related issues from the mock data get the link to context and generate any case like if all these four panels are working using that mock data those mock data particularly json files only so we are trying to use that but as far as a full implementation is concerned we are particularly thinking of the design phase only we are trying to finalize this and we discussed this in the previous meeting that we will be discussing sequence diagrams and then if the sequence diagram finalizes then we will be further moving to the implementation phase.

Yeah what I was trying to ask is like you know to validate the design whether this is feasible or not have you done some small prototypes that was the question not for the complete implementation start as such have you done the required design validations? Yes we do try to link to external systems we try to run docker and use Kafka Redis run those docker containers and we try to fetch data from those json files and make sure that these panels are working so we have tried to generate it generate that we have tried to write those connector classes through which we will be interacting with the external system we are in practice like we are trying it simultaneously but if there is something to demonstrate end-to-end today we are not ready with that. Yeah that's fine so it's not meant for demo purpose for your own design validation that would be useful so if there is any such validation code you could choose to check in that as well at the github and then proceed towards the actual implementation yeah yes yes and we are doing this we are working on that and we will validate the psu also. Okay so in the beginning we used to have that issue where you know we were trying to persist locally with all that work database information so I hope all of that is now sorted and we have a good we have a design where we are dynamically fetching the information on demand and whatever minimal is required we are trying to persist only those aspects is that correct? Yes we are we are pulling that live bug list like if we could refer back to the previous slide so everything is fetched live from the external systems we are not storing anything we will just be storing the context for a temporary basis just to ensure our system is fault-tolerant but everything we are fetching is live from the external systems.

Okay so fine so all of this is the design that you have demonstrated here is it checked in in the github for us to be offline as well? Yes github it has been posted to github and it has

been discussed in the previous meetings and today we have completed with the sequence diagrams also we will be posting the sequence diagrams today. Okay anything else Rajesh and Diwakar? No I think I'm good yeah I'm good just to ensure if the end user sees this bootstrats tool are we addressing all the issues? Sorry what is it? Like if we seeing this UI mockup and the design decisions we made so are we addressing all the issues we just need to confirm that? Yeah it looks good to me like based on whatever you have just explained I think it's looking good. So you could proceed with the like checking all of those design validation code that you have written you might have written and if not do that due diligence to see like to make sure that the feasibility part is taken care and then you can get started with the actual implementation and you can plan out the implementation and share the sort of a roadmap with us in terms of the estimates and how it has been distributed between you between all of you.

So yes and what would be the timeline for this project closure like we will have to plan accordingly. Madhukar I think we got a communication from the university team from from HPE that this is end of June or something sorry I'm going to miss the dates but yeah usually it ends by the month end of June I think so we are a few they'll have a month's time I think now. Yeah sure and the final implementation would consist everything working right we will be seeking those open source repositories fetching data from them and everything will be working so that we find as a final presentation.

Yeah you write down what it will be what would be your data sources as we said like those open source projects that you will be choosing document each of the design decisions and all of these aspects in the data and data pages and then you can get started and write down the roadmap or the plan of implementation along with the estimates and share it with us before you start. Yeah sure I think that's all we have for the meeting. Thank you.